

Chín bài giảng về bài toán cái túi

Cui Tianyi

Bản nguồn: 2012-05-08

Nguồn gốc: Knapsack Problem: Nine Lectures 2.0 beta1.2

Dynamic Programming / Knapsack Nine Lectures - Cui Tianyi

Abstract

Bài viết này hệ thống hóa các biến thể thường gặp của bài toán cái túi trong quy hoạch động: cái túi 01, cái túi hoàn toàn, cái túi nhiều bản sao, mô hình kết hợp, hai loại chi phí, phân nhóm, phụ thuộc, vật phẩm tổng quát và các cách hồi biến dạng như in phương án, đếm phương án hoặc tìm nghiệm tốt thứ K . Điểm trọng tâm không chỉ là công thức, mà là cách nhìn ra trạng thái, hiểu ý nghĩa của chuyển trạng thái và dùng đúng thứ tự lặp để giảm bộ nhớ.

Contents

1	Bài toán cái túi 01	1
1.1	Đề bài	1
1.2	Ý tưởng cơ bản	1
1.3	Giảm bộ nhớ	2
1.4	Khởi tạo	2
1.5	Một tối ưu hằng số	2
1.6	Tóm tắt	2
2	Bài toán cái túi hoàn toàn	3
2.1	Đề bài	3
2.2	Ý tưởng cơ bản	3
2.3	Loại vật phẩm bị trội	3
2.4	Chuyển thành cái túi 01	3
2.5	Thuật toán $\mathcal{O}(NV)$	3
2.6	Tóm tắt	4
3	Bài toán cái túi nhiều bản sao	4
3.1	Đề bài	4
3.2	Công thức cơ bản	4
3.3	Phân rã thành vật phẩm 01	4
3.4	Bài toán khả thi trong $\mathcal{O}(NV)$	4
3.5	Tóm tắt	5

4	Kết hợp ba loại cái túi	5
4.1	Bài toán	5
4.2	Trộn 01 và hoàn toàn	5
4.3	Thêm loại nhiều bản sao	5
5	Cái túi với hai loại chi phí	6
5.1	Bài toán	6
5.2	Thuật toán	6
5.3	Giới hạn số lượng vật phẩm	6
5.4	Góc nhìn số phức nguyên	6
6	Bài toán cái túi phân nhóm	6
6.1	Bài toán	6
6.2	Thuật toán	6
6.3	Tóm tắt	7
7	Cái túi có quan hệ phụ thuộc	7
7.1	Dạng đơn giản	7
7.2	Thuật toán	7
7.3	Dạng tổng quát hơn	7
7.4	Tóm tắt	7
8	Vật phẩm tổng quát	8
8.1	Định nghĩa	8
8.2	Tổng của hai vật phẩm tổng quát	8
8.3	Ý nghĩa	8
9	Các biến thể trong cách hỏi	8
9.1	In phương án	8
9.2	In phương án tối ưu có thứ tự từ điển nhỏ nhất	9
9.3	Đếm số phương án	9
9.4	Đếm số phương án tối ưu	9
9.5	Nghiệm tốt thứ hai và nghiệm tốt thứ K	9
9.6	Tóm tắt	10

1 Bài toán cái túi 01

1.1 Đề bài

Có N vật phẩm và một cái túi dung lượng V . Chọn vật phẩm i sẽ tốn chi phí C_i và nhận được giá trị W_i . Mỗi vật phẩm có nhiều nhất một bản. Hãy chọn một tập vật phẩm sao cho tổng chi phí không vượt quá V và tổng giá trị là lớn nhất.

1.2 Ý tưởng cơ bản

Đây là dạng nền tảng nhất. Với mỗi vật phẩm chỉ có hai chiến lược: lấy hoặc không lấy. Đặt

$$F[i, v]$$

là giá trị lớn nhất có thể đạt được khi chỉ xét i vật phẩm đầu và dùng dung lượng đúng bằng v . Khi xét vật phẩm i , nếu không lấy nó thì trạng thái đến từ $F[i - 1, v]$; nếu lấy nó thì phần còn lại là $v - C_i$, nên giá trị là $F[i - 1, v - C_i] + W_i$. Do đó

$$F[i, v] = \max\{F[i - 1, v], F[i - 1, v - C_i] + W_i\}.$$

Pseudocode hai chiều:

```
F[0, 0..V] <- 0
for i <- 1 to N
  for v <- C_i to V
    F[i, v] <- max(F[i-1, v], F[i-1, v-C_i] + W_i)
```

Công thức này là hạt nhân của hầu hết các biến thể cái túi. Khi gặp biến thể mới, trước hết hãy hỏi: chiến lược đối với vật phẩm hiện tại là gì, và sau khi chọn chiến lược đó ta quay về bài toán con nào.

1.3 Giảm bộ nhớ

Cách trên dùng $\mathcal{O}(NV)$ thời gian và $\mathcal{O}(NV)$ bộ nhớ. Thời gian thường khó giảm thêm, nhưng bộ nhớ có thể giảm còn $\mathcal{O}(V)$. Dùng mảng một chiều $F[0..V]$ và cập nhật theo thứ tự giảm của v :

```
F[0..V] <- 0
for i <- 1 to N
  for v <- V downto C_i
    F[v] <- max(F[v], F[v-C_i] + W_i)
```

Thứ tự giảm là chi tiết quyết định. Khi tính $F[v]$ cho vật phẩm i , ta phải dùng giá trị $F[v - C_i]$ của vòng trước, tức là trạng thái chưa lấy vật phẩm i . Nếu lặp tăng, $F[v - C_i]$ có thể đã được cập nhật bởi chính vật phẩm i , làm cho một vật phẩm bị lấy nhiều lần.

Vì đoạn cập nhật này xuất hiện nhiều lần, ta tách thành thủ tục:

```
def ZeroOnePack(F, C, W)
  for v <- V downto C
    F[v] <- max(F[v], F[v-C] + W)
```

Khi đó bài toán 01 viết gọn:

```
F[0..V] <- 0
for i <- 1 to N
  ZeroOnePack(F, C_i, W_i)
```

1.4 Khởi tạo

Các đề cái túi tối ưu thường có hai cách hỏi. Nếu yêu cầu *đúng bằng* dung lượng V , ta khởi tạo

$$F[0] = 0, \quad F[1..V] = -\infty.$$

Như vậy chỉ dung lượng 0 là đạt được khi chưa chọn gì; các dung lượng khác chưa có nghiệm hợp lệ.

Nếu không yêu cầu lấp đầy túi mà chỉ cần tổng chi phí không vượt quá V , ta khởi tạo toàn bộ $F[0..V]$ bằng 0. Khi đó phương án “không chọn gì” là hợp lệ cho mọi giới hạn dung lượng và có giá trị 0. Quy tắc này dùng được cho cả các biến thể phía sau.

1.5 Một tối ưu hằng số

Trong vòng lặp

```
for i <- 1 to N
  for v <- V downto C_i
```

biên dưới đôi khi có thể nâng lên bằng cách quan sát tổng chi phí các vật phẩm còn lại. Đây chỉ là tối ưu hằng số, không thay đổi bậc phức tạp, nhưng nhắc ta rằng cách cắt phạm vi trạng thái hợp lệ cũng quan trọng như công thức chuyển.

1.6 Tóm tắt

Cái túi 01 chứa ba ý tưởng cơ bản: định nghĩa trạng thái theo tiền tố vật phẩm, chuyển theo lựa chọn lấy hoặc không lấy, và giảm bộ nhớ bằng thứ tự lặp giảm. Nhiều bài cái túi khác chỉ là biến dạng của ba ý tưởng này.

2 Bài toán cái túi hoàn toàn

2.1 Đề bài

Có N loại vật phẩm, mỗi loại có vô hạn bản. Loại i có chi phí C_i và giá trị W_i . Hãy chọn các vật phẩm sao cho tổng chi phí không vượt quá V và tổng giá trị là lớn nhất.

2.2 Ý tưởng cơ bản

Với mỗi loại vật phẩm, chiến lược không còn chỉ là lấy hoặc không lấy, mà là lấy $0, 1, 2, \dots, \lfloor V/C_i \rfloor$ bản. Nếu vẫn đặt $F[i, v]$ là giá trị tốt nhất từ i loại đầu, ta có

$$F[i, v] = \max\{F[i-1, v - kC_i] + kW_i \mid 0 \leq kC_i \leq v\}.$$

Công thức này đúng nhưng tính trực tiếp khá chậm, vì mỗi trạng thái lại phải liệt kê số bản k .

2.3 Loại vật phẩm bị trội

Nếu hai loại i, j thỏa $C_i \leq C_j$ và $W_i \geq W_j$, thì loại j không bao giờ cần dùng: mọi phương án dùng j đều có thể thay bằng i mà không tệ hơn. Ta có thể loại bỏ các vật phẩm như vậy. Với dữ liệu ngẫu nhiên, tiên lý này thường giảm đáng kể số vật phẩm, dù không cải thiện độ phức tạp tệ nhất.

2.4 Chuyển thành cái túi 01

Có thể biến mỗi loại vật phẩm thành nhiều vật phẩm 01. Cách ngây thơ là tạo $\lfloor V/C_i \rfloor$ bản giống nhau. Cách tốt hơn là dùng phân rã nhị phân: tạo các gói có hệ số $1, 2, 4, \dots$. Vì mọi số bản đều biểu diễn được bằng tổng các lũy thừa hai, mỗi loại chỉ cần $\mathcal{O}(\log(V/C_i))$ gói.

2.5 Thuật toán $\mathcal{O}(NV)$

Cái túi hoàn toàn có cách đơn giản hơn. Dùng mảng một chiều và lặp v theo thứ tự tăng:

```
F[0..V] <- 0
for i <- 1 to N
  for v <- C_i to V
    F[v] <- max(F[v], F[v-C_i] + W_i)
```

Khác biệt duy nhất so với 01 là thứ tự lặp. Ở đây khi xét thêm một bản của loại i , ta *muốn* được dùng trạng thái đã có thể chứa loại i , vì mỗi loại có thể lấy nhiều lần. Do đó $F[v - C_i]$ phải là giá trị trong cùng vòng lặp, và ta lặp tăng.

Thủ tục tương ứng:

```
def CompletePack(F, C, W)
  for v <- C to V
    F[v] <- max(F[v], F[v-C] + W)
```

Công thức hai chiều tương đương là

$$F[i, v] = \max\{F[i - 1, v], F[i, v - C_i] + W_i\}.$$

Công thức này cũng giải thích trực tiếp vì sao lặp tăng là đúng.

2.6 Tóm tắt

Cái túi hoàn toàn giúp phân biệt rõ hai trạng thái “vòng trước” và “cùng vòng”. Chỉ cần đảo chiều lặp của v , ta chuyển từ mỗi vật phẩm dùng một lần sang mỗi loại dùng vô hạn lần.

3 Bài toán cái túi nhiều bản sao

3.1 Đề bài

Có N loại vật phẩm và dung lượng V . Loại i có tối đa M_i bản, mỗi bản tốn C_i và được W_i . Hãy tối đa hóa tổng giá trị với tổng chi phí không vượt quá V .

3.2 Công thức cơ bản

Mỗi loại có $M_i + 1$ chiến lược: lấy 0 đến M_i bản. Do đó

$$F[i, v] = \max\{F[i - 1, v - kC_i] + kW_i \mid 0 \leq k \leq M_i\}.$$

Cách tính trực tiếp có độ phức tạp $\mathcal{O}(V \sum_i M_i)$.

3.3 Phân rã thành vật phẩm 01

Thay vì tạo M_i vật phẩm giống nhau, ta phân rã số lượng theo nhị phân. Các hệ số là

$$1, 2, 4, \dots, 2^{k-1}, M_i - 2^k + 1,$$

với k lớn nhất sao cho phần cuối còn dương. Ví dụ $M_i = 13$ được tách thành 1, 2, 4, 6. Mỗi gói là một vật phẩm 01 có chi phí và giá trị nhân với hệ số. Tổng các hệ số đúng bằng M_i , và mọi số từ 0 đến M_i đều biểu diễn được bằng một số gói.

Thủ tục xử lý một loại nhiều bản sao:

```
def MultiplePack(F, C, W, M)
  if C * M >= V
    CompletePack(F, C, W)
    return
  k <- 1
  while k < M
    ZeroOnePack(F, k*C, k*W)
    M <- M - k
    k <- 2*k
  ZeroOnePack(F, M*C, M*W)
```

Điều kiện $C \cdot M \geq V$ nghĩa là giới hạn số bản không còn tác dụng trong phạm vi dung lượng V , nên loại đó có thể xử lý như cái túi hoàn toàn.

3.4 Bài toán khả thi trong $\mathcal{O}(NV)$

Nếu chỉ hỏi có thể lấp đầy từng dung lượng hay không, không xét giá trị, cái túi nhiều bản sao còn có một cách $\mathcal{O}(NV)$ đơn giản. Đặt $F[i, j]$ là số bản còn lại nhiều nhất của loại i sau khi dùng i loại đầu để đạt dung lượng j ; nếu không đạt được thì bằng -1 . Chuyển:

```
F[0, 1..V] <- -1
F[0, 0] <- 0
for i <- 1 to N
  for j <- 0 to V
    if F[i-1, j] >= 0
      F[i, j] <- M_i
    else
      F[i, j] <- -1
  for j <- 0 to V-C_i
    if F[i, j] > 0
      F[i, j+C_i] <- max(F[i, j+C_i], F[i, j] - 1)
```

Ý nghĩa là: nếu dung lượng j đã đạt được trước khi xét loại i , ta có đầy đủ M_i bản để tiếp tục dùng. Nếu trạng thái hiện tại còn bản của loại i , ta có thể dùng thêm một bản và giảm số còn lại.

3.5 Tóm tắt

Điểm chính của dạng này là “tách vật phẩm”. Phân rã nhị phân đưa độ phức tạp từ $\mathcal{O}(V \sum_i M_i)$ xuống $\mathcal{O}(V \sum_i \log M_i)$. Với bài chỉ cần khả thi, có thể dùng trạng thái “số bản còn lại” để đạt $\mathcal{O}(NV)$.

4 Kết hợp ba loại cái túi

4.1 Bài toán

Một số vật phẩm chỉ lấy được một lần, một số lấy được vô hạn lần, một số có giới hạn số bản. Hãy giải trong cùng một mô hình.

4.2 Trộn 01 và hoàn toàn

Vì hai thủ tục chỉ khác hướng lặp, ta chọn hướng theo loại vật phẩm:

```
for i <- 1 to N
  if item i is 01
    for v <- V downto C_i
      F[v] <- max(F[v], F[v-C_i] + W_i)
  else if item i is complete
    for v <- C_i to V
      F[v] <- max(F[v], F[v-C_i] + W_i)
```

4.3 Thêm loại nhiều bản sao

Khi có cả ba loại, cách viết rõ ràng nhất là dùng ba thủ tục đã tách:

```
for i <- 1 to N
  if item i is 01
    ZeroOnePack(F, C_i, W_i)
  else if item i is complete
    CompletePack(F, C_i, W_i)
  else if item i is multiple
    MultiplePack(F, C_i, W_i, M_i)
```

Sức mạnh ở đây nằm ở việc trừu tượng hóa. Khi hiểu từng thủ tục và sự khác nhau giữa chúng, bài toán tưởng phức tạp chỉ còn là lựa chọn đúng thao tác cho từng vật phẩm.

5 Cái túi với hai loại chi phí

5.1 Bài toán

Mỗi vật phẩm cần trả đồng thời hai chi phí. Vật phẩm i có hai chi phí C_i, D_i , hai giới hạn dung lượng là V, U , và giá trị là W_i . Hãy tối đa hóa tổng giá trị.

5.2 Thuật toán

Chỉ cần tăng thêm một chiều trạng thái. Với vật phẩm 01:

$$F[i, v, u] = \max\{F[i-1, v, u], F[i-1, v-C_i, u-D_i] + W_i\}.$$

Khi giảm bộ nhớ, dùng mảng $F[v, u]$. Nếu vật phẩm là 01, cả v và u lặp giảm; nếu là hoàn toàn thì lặp tăng; nếu là nhiều bản sao thì dùng phân rã nhị phân hoặc thủ tục tương ứng.

5.3 Giới hạn số lượng vật phẩm

Ràng buộc “chọn nhiều nhất U vật phẩm” cũng là một chi phí thứ hai: mỗi vật phẩm có chi phí số lượng bằng 1. Đặt $F[v, u]$ là giá trị tốt nhất khi dùng chi phí chính v và chọn tối đa u vật phẩm, rồi cập nhật theo loại vật phẩm.

5.4 Góc nhìn số phức nguyên

Có thể xem chi phí hai chiều như chi phí trên miền số phức nguyên: một đơn vị chi phí là cặp (v, u) . Khi đó nhiều kỹ thuật của cái túi một chiều vẫn giữ nguyên, chỉ là miền trạng thái rộng hơn.

6 Bài toán cái túi phân nhóm

6.1 Bài toán

Có N vật phẩm, dung lượng V . Các vật phẩm được chia thành K nhóm; trong mỗi nhóm, nhiều nhất được chọn một vật phẩm. Hãy tối đa hóa tổng giá trị.

6.2 Thuật toán

Mỗi nhóm tương ứng với một tập chiến lược loại trừ nhau: không chọn gì, hoặc chọn đúng một vật phẩm trong nhóm. Đặt $F[k, v]$ là giá trị tốt nhất sau k nhóm với chi phí v , ta có

$$F[k, v] = \max\{F[k-1, v], F[k-1, v-C_i] + W_i \mid i \text{ thuộc nhóm } k\}.$$

Dùng mảng một chiều:

```
for k <- 1 to K
  for v <- V downto 0
    for each item i in group k
      F[v] <- max(F[v], F[v-C_i] + W_i)
```

Thứ tự ba vòng lặp bảo đảm trong một nhóm chỉ có thể chọn nhiều nhất một vật phẩm. Trong từng nhóm cũng có thể loại bỏ vật phẩm bị trội như ở cái túi hoàn toàn.

6.3 Tóm tắt

Phân nhóm là mô hình cho các lựa chọn loại trừ nhau. Nhiều bài có ràng buộc phụ thuộc hoặc nhiều phương án cục bộ có thể quy về các nhóm vật phẩm.

7 Cái túi có quan hệ phụ thuộc

7.1 Dạng đơn giản

Vật phẩm i phụ thuộc vào vật phẩm j nghĩa là nếu chọn i thì bắt buộc phải chọn j . Trước hết xét trường hợp đơn giản: không có vật phẩm vừa phụ thuộc vào vật phẩm khác vừa bị vật phẩm khác phụ thuộc, và mỗi vật phẩm phụ thuộc nhiều nhất một vật phẩm.

7.2 Thuật toán

Gọi các vật phẩm không phụ thuộc ai là **vật chính**, còn các vật phẩm phụ thuộc vào một vật chính là **phụ kiện**. Nếu xét một vật chính cùng các phụ kiện của nó, các chiến lược hợp lệ là: không chọn gì; chọn vật chính; chọn vật chính kèm một số phụ kiện. Những chiến lược này loại trừ nhau, nên chúng tạo thành một nhóm trong bài toán phân nhóm.

Nếu vật chính có n phụ kiện, số chiến lược thô là $2^n + 1$, quá lớn. Ta có thể làm tốt hơn: trước hết chạy một bài 01 cho tập phụ kiện để biết với mỗi chi phí $0..V - C_k$, giá trị phụ kiện tốt nhất là bao nhiêu. Sau đó vật chính k cùng phụ kiện của nó được biến thành một nhóm gồm các vật phẩm tổng hợp theo từng chi phí, mỗi vật phẩm có giá trị bằng giá trị phụ kiện tốt nhất cộng với W_k .

7.3 Dạng tổng quát hơn

Nếu quan hệ phụ thuộc tạo thành một rừng, phụ kiện cũng có thể có phụ kiện riêng. Khi đó ta xử lý từ lá lên gốc. Mỗi cây con được xem như một vật phẩm tổng quát; trước khi tính nút cha, ta đã biết các “vật phẩm tổng quát” của các con và kết hợp chúng bằng quy hoạch động trên cây.

7.4 Tóm tắt

Bài cái túi phụ thuộc là một cách nhìn khác của cái túi phân nhóm và quy hoạch động trên cây. Điều quan trọng là gói các chiến lược hợp lệ của một cụm phụ thuộc thành một đối tượng để ghép với các cụm khác.

8 Vật phẩm tổng quát

8.1 Định nghĩa

Một vật phẩm tổng quát không có chi phí và giá trị cố định. Thay vào đó, nếu cấp cho nó chi phí v , nó trả về giá trị $h(v)$. Với dung lượng V , có thể xem vật phẩm tổng quát là một hàm trên các số nguyên $0..V$, hoặc đơn giản là một mảng $h[0..V]$.

Một vật phẩm 01 có chi phí c , giá trị w tương ứng với hàm chỉ có $h(c) = w$ là đáng kể. Một vật phẩm hoàn toàn có $h(v) = w \cdot v/c$ khi c chia hết v . Một vật phẩm nhiều bản sao có cùng dạng nhưng chỉ hợp lệ khi $v/c \leq m$. Một nhóm vật phẩm cũng là vật phẩm tổng quát: với mỗi chi phí v , giữ giá trị lớn nhất trong nhóm có chi phí đó.

8.2 Tổng của hai vật phẩm tổng quát

Cho hai vật phẩm tổng quát h và l . Nếu cấp tổng chi phí v cho cả hai, ta chia v thành k và $v - k$:

$$f(v) = \max\{h(k) + l(v - k) \mid 0 \leq k \leq v\}.$$

Hàm f cũng là một vật phẩm tổng quát, gọi là tổng của h và l . Phép cộng này mất $\mathcal{O}(V^2)$ nếu làm trực tiếp.

Trong một bài cái túi, thay hai vật phẩm tổng quát bằng tổng của chúng không làm đổi đáp án. Vì vậy giải một bài cái túi có thể được hiểu là tính tổng của tất cả các vật phẩm tổng quát rồi lấy giá trị tốt nhất trên miền $0..V$.

8.3 Ý nghĩa

Cách nhìn bằng vật phẩm tổng quát nén mọi điều kiện của bài toán thành một hàm: với mỗi dung lượng v , giá trị tốt nhất có thể đạt được là bao nhiêu. Nhiều mô hình như phân nhóm, phụ thuộc và cây phụ thuộc đều trở nên thống nhất dưới phép cộng các vật phẩm tổng quát.

9 Các biến thể trong cách hỏi

Các phần trước đều hỏi giá trị lớn nhất dưới giới hạn dung lượng. Khi cách hỏi thay đổi, nếu đã hiểu công thức chuyển thì thường chỉ cần thay đổi trạng thái hoặc phép toán.

9.1 In phương án

Để in một phương án tối ưu, ghi lại chiến lược tạo ra mỗi trạng thái. Với 01 cái túi, dùng mảng $G[i, v]$: $G[i, v] = 0$ nếu $F[i, v]$ đến từ $F[i - 1, v]$, và $G[i, v] = 1$ nếu đến từ $F[i - 1, v - C_i] + W_i$. Sau đó lần ngược:

```
i <- N
v <- V
while i > 0
  if G[i, v] = 0
    print "item i is not chosen"
  else
    print "item i is chosen"
    v <- v - C_i
  i <- i - 1
```

Cũng có thể không lưu G , mà so sánh trực tiếp các giá trị F khi truy vết.

9.2 In phương án tối ưu có thứ tự từ điển nhỏ nhất

Nếu cần phương án tối ưu có thứ tự từ điển nhỏ nhất theo dãy chọn các vật phẩm $1..N$, phải cẩn thận với định nghĩa trạng thái. Một cách đơn giản là đảo thứ tự đánh số vật phẩm trước khi chạy DP. Khi truy vết, nếu cả hai lựa chọn “không lấy” và “lấy” đều cho giá trị tối ưu, ưu tiên lấy vật phẩm trong thứ tự đã đảo, rồi đổi chỉ số về ban đầu. Nhờ vậy quyết định sớm hơn theo chỉ số gốc được ưu tiên đúng cách.

9.3 Đếm số phương án

Nếu cần đếm số cách đạt một dung lượng, thường thay phép max bằng phép cộng. Ví dụ với cái túi hoàn toàn, số cách thỏa

$$F[i, v] = F[i - 1, v] + F[i, v - C_i],$$

với điều kiện đầu $F[0, 0] = 1$. Lý do là công thức chuyển đã liệt kê hết các khả năng tạo thành trạng thái.

9.4 Đếm số phương án tối ưu

Khi cần vừa tối ưu giá trị vừa đếm số phương án tối ưu, lưu hai mảng. $F[i, v]$ là giá trị tốt nhất, $G[i, v]$ là số phương án đạt giá trị đó:

```

G[0, 0] <- 1
for i <- 1 to N
  for v <- 0 to V
    F[i, v] <- max(F[i-1, v], F[i-1, v-C_i] + W_i)
    G[i, v] <- 0
    if F[i, v] = F[i-1, v]
      G[i, v] <- G[i, v] + G[i-1, v]
    if F[i, v] = F[i-1, v-C_i] + W_i
      G[i, v] <- G[i, v] + G[i-1, v-C_i]

```

Nếu hai chiến lược cùng đạt giá trị tối ưu, số phương án của cả hai phía đều được cộng vào.

9.5 Nghiệm tốt thứ hai và nghiệm tốt thứ K

Nếu bài tối ưu có công thức DP, phiên bản tìm nghiệm tốt thứ K thường có thể xử lý bằng cách biến mỗi trạng thái thành một hàng đợi có thứ tự gồm K giá trị tốt nhất. Với 01 cái túi, thay vì một số $F[i, v]$, lưu

$$F[i, v, 1..K],$$

trong đó phần tử thứ k là giá trị tốt thứ k ở trạng thái đó. Công thức

$$F[i, v] = \max\{F[i-1, v], F[i-1, v-C_i] + W_i\}$$

trở thành thao tác trộn hai dãy đã sắp giảm:

$$F[i-1, v, 1..K] \quad \text{và} \quad F[i-1, v-C_i, 1..K] + W_i.$$

Giữ lại K phần tử đầu sau khi trộn. Mỗi trạng thái tốn $\mathcal{O}(K)$, nên tổng thời gian là $\mathcal{O}(NVK)$. Nếu đề xem hai phương án cùng giá trị là một nghiệm, cần loại giá trị trùng khi duy trì dãy.

9.6 Tóm tắt

Các cách hỏi khác nhau không làm mất đi bản chất của bài toán. Khi đã hiểu trạng thái và chuyển trạng thái của phiên bản tối ưu, ta có thể đổi phép toán, thêm thông tin truy vết hoặc lưu nhiều giá trị tốt nhất để xử lý yêu cầu mới.

Lời kết

Bài toán cái túi không chỉ là một mẫu quy hoạch động đơn lẻ. Nó là một họ mô hình có cùng lõi: biểu diễn ràng buộc bằng dung lượng, biểu diễn lựa chọn bằng chuyển trạng thái, rồi tối ưu thứ tự lặp và cách gói vật phẩm. Nắm chắc các dạng cơ bản giúp ta nhận ra và xử lý được nhiều biến thể hơn trong thi lập trình.